Архитектура вычислительных систем 2025

Домашнее задание №4

Шептяков Артём
Группа: Б01-207
Почта: sheptiakov.ad@phystech.edu

Политики

LRU

LRU (Least Recently Used) — это классическая политика замещения кэша, в основе которой лежит вытеснение данных, к которым дольше всего не было обращений. LRU оперирует принципом временной локальности, в предположении, что недавно использованные данные с большей вероятностью понадобятся снова. Каждый доступ к строке кэша обновляет её позицию в списке актуальности, а при необходимости замещения удаляется самая «старая» запись. Несмотря на простоту, LRU может работать неэффективно при поиске по сетке или в циклах, где данные используются однократно, но занимают кэш, вытесняя полезные строки.

SHiP

SHiP (Signature-based Hit Predictor) — это политика замещения кэша, в основе которой лежит предсказание вероятности повторного обращения к данным на основе их уникальных сигнатур (например, адреса памяти, программного счетчика или истории инструкций). SHiP использует таблицу счетчиков (SHCT) для обучения: если данные с определенной сигнатурой ранее получали попадания в кэш, новые данные с такой же сигнатурой сохраняются дольше, в противном случае они быстрее замещаются. Это позволяет улучшить производительность кэша на 9–12% по сравнению с LRU и другими современными политиками, сохраняя низкие аппаратные затраты.

SPP

SPP (Signature Path Prefetcher) — это предварительный механизм выборки данных, в основе которого лежит предсказание будущих обращений к памяти на основе сжатых сигнатур истории доступа. SPP использует таблицу сигнатур (ST) для отслеживания шаблонов доступа внутри 4КБ страниц и глобальную таблицу паттернов (PT) для предсказания следующих дельт (разниц адресов). Ключевая особенность SPP — адаптивное управление глубиной предварительной выборки на основе "уверенности пути"(path confidence), что позволяет балансировать между покрытием и точностью. Кроме того, SPP эффективно обрабатывает переходы между страницами, сохраняя шаблоны доступа через границы страниц с помощью глобального регистра истории (GHR). В сравнении с другими механизмами SPP демонстрирует лучшее среднее ускорение при минимальных аппаратных накладных расходах.

Анализ

Для анализа были выбраны следующие трассы:

- 437.leslie3d-134B.champsimtrace.xz
- 453.povray-252B.champsimtrace.xz
- 625.x264_s-12B.champsimtrace.xz

437.leslie3d-134B.champsimtrace.xz

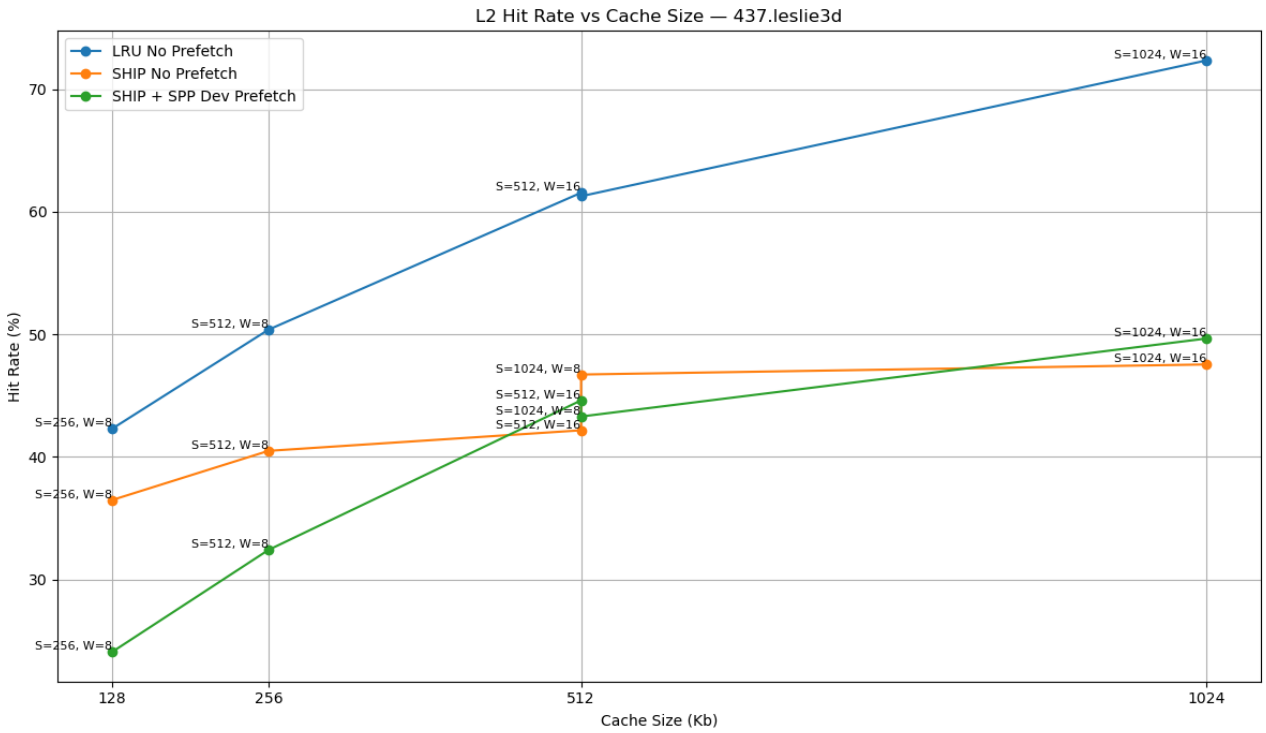

Рис. 1: 437.leslie3d

Эта трасса показывает, как ведёт себя кэш при моделировании гидродинамических процессов. В целом, политика замещения LRU без префетчера (предвыборки) работает лучше всего: с увеличением размера кэша hit rate растёт и достигает максимума в 72,35% при конфигурации 1024 sets, 16 ways. Это говорит о том, что чем больше кэш, тем эффективнее он работает, по крайней мере, для этой политики.

SHIP без префетчера работает хуже, чем LRU, особенно на маленьких кэшах. Например, при 256 sets, 8 ways hit rate составляет всего 36,49%, тогда как у LRU уже 42,31%. Но с увеличением размера кэша разница между политиками становится меньше, хотя LRU всё равно остаётся впереди, SHIP тоже реагирует на увеличение кэша, но не так эффективно.

Когда включается префетчер SPP Dev, ситуация немного усложняется. С одной стороны, hit rate падает, особенно на малых кэшах, потому что предвыборка может загружать лишние данные, которые потом не используются. С другой стороны, на больших кэшах (например, 1024sets, 16 ways) hit rate поднимается до 49,66%, то есть эффект от префетчинга начинает проявляться, хотя и не сильно. Таким образом, для этой трассы префетчинг помогает только при больших кэшах, а в целом он не даёт такого выигрыша, как просто увеличение объёма кэша.

Для данной трассы самая лучшая производительность достигается с LRU и большим кэшем. SHIP работает хуже, но разрыв уменьшается при увеличении размера кэша. Префетчинг почти всегда снижает hit rate, если кэш маленький, но может дать небольшой прирост на больших кэшах.

453.povray-252B.champsimtrace.xz

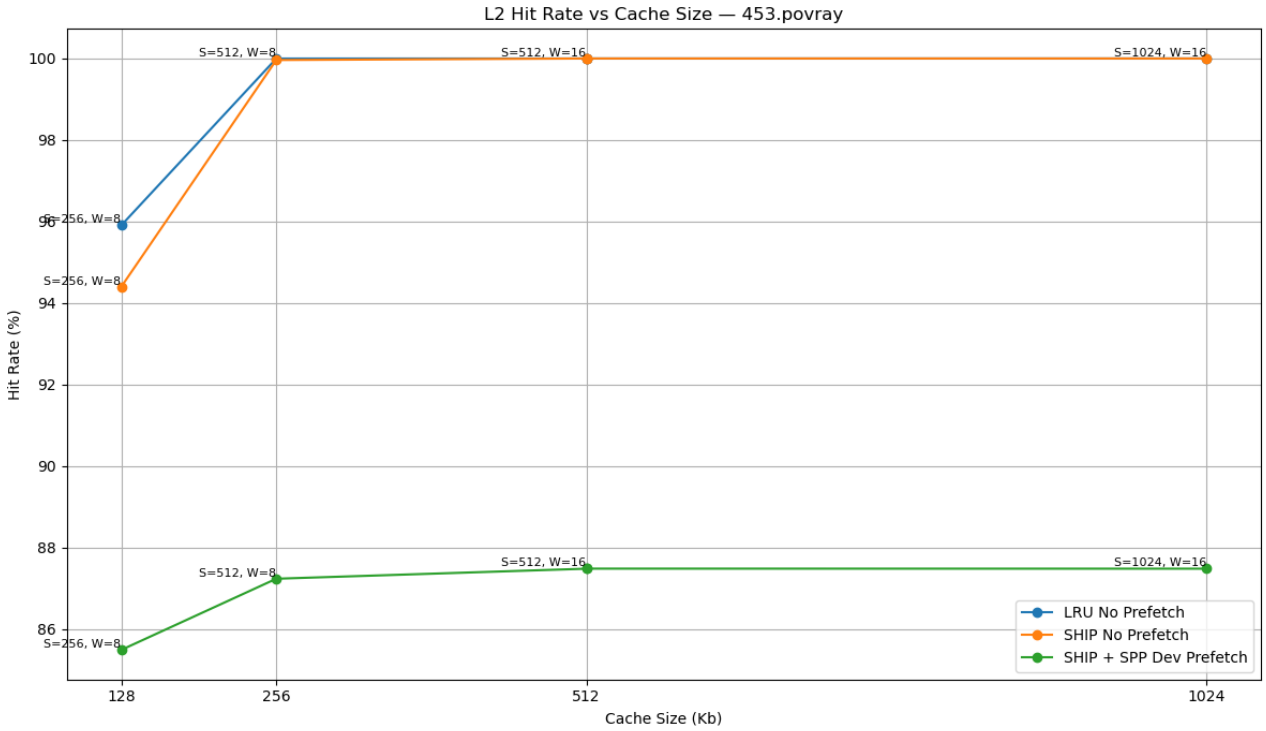

Рис. 2: 453.povray

Данная трасса связана с рендерингом изображений и обладает хорошей локальностью данных. Для всех конфигураций hit rate получился очень высоким — почти 100% для LRU и SHIP без префетчера даже на самых маленьких кэшах. Например, при 256 sets, 8 ways политика LRU показала 95,9% попаданий, а уже при 512 sets, 8 ways hit rate вырос до 99,99

SHIP без префетчера тоже показал отличные результаты — почти такие же, как у LRU. Разница между политиками практически неощутима, и это говорит о том, что для таких "дружелюбных"к кэшу приложений выбор политики замещения не играет большой роли.

Однако в случае с префетчером SPP Dev, hit rate заметно упал — до 85–87% на маленьких кэшах и до 87,5% на больших. Это странно, потому что можно было ожидать, что префетчинг поможет, но вместо этого он добавляет лишних промахов, вероятно, потому что загружает слишком много ненужных данных.

Эта трасса хорошо работает с любым кэшом, даже маленьким. Политика замещения не играет большой роли, потому что данные повторяются, рефетчинг в данном случае вредит производительности, так как предположительно тратит место в кэше на данные, которые не будут использованы.

625.x264_s-12B.champsimtrace.xz

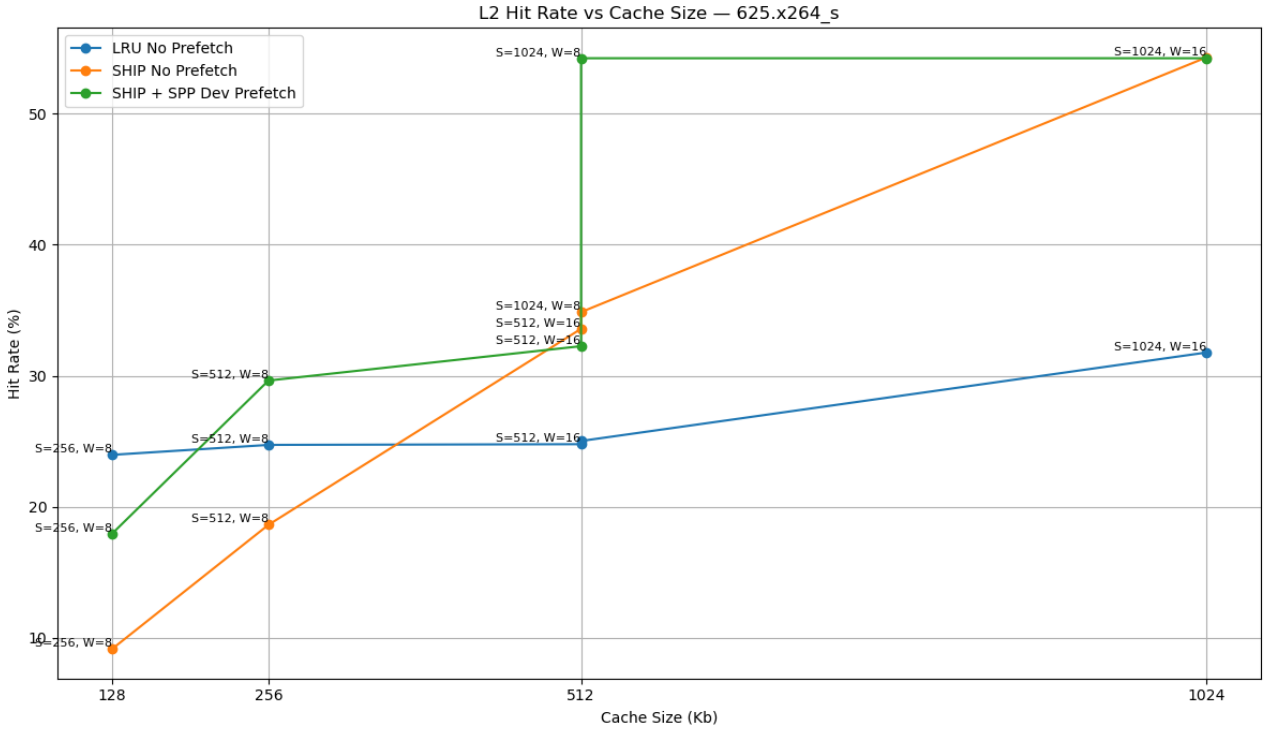

Рис. 3: 625.x264_s

В противовес прошлой трассе в данном случае кодирование видео обладает плохой локальностью данных — другими словами, процессор часто обращается к новым данным, поэтому кэш заполняется быстро и часто возникают промахи. Для этой трассы hit rate ниже, чем для двух предыдущих.

LRU без префетчера работает стабильно, но даёт малый прирост при увеличении объёма: даже при самом большом кэше (1024 sets, 16 ways) hit rate равен лишь 31,77%. Это говорит о том, что даже большой кэш не сильно помогает при плохой временной локальности.

SHIP без префетчера работает хуже всего на маленьких кэшах: например, при 256 sets, 8 ways hit rate всего 9,16%, но с увеличением кэша SHIP резко улучшается — при 1024 sets, 16 ways hit rate уже 54,28%, то есть почти в два раза лучше, чем у LRU. Таким образом, SHIP оказался более чувствителен к увеличению размера кэша, чем LRU.

Самый большой прирост дал префетчер SPP Dev: при 1024 sets, 16 ways hit rate достиг 54,22%, что почти совпадает с SHIP без префетчера. Это означает, что префетчинг действительно помогает, когда данные плохо локализованы и кэш часто переполняется. В данном случае предвыборка данных оказывается полезной, потому что позволяет заранее подготовить нужные данные, даже если они не повторяются.

Для данной трассы префетчинг и большой кэш работают лучше всего. SHIP при этом становится эффективнее LRU, если кэш достаточно большой. LRU без префетчера работает стабильнее, но не так эффективно, как SHIP с префетчингом на больших кэшах.

Заключение

Политика замещения LRU работает лучше всего в большинстве случаев, особенно когда данные имеют хорошую локальность (например, в povray). SHIP, хотя и уступает LRU на маленьких кэшах, может догнать его или даже обогнать, если кэш достаточно большой. Это особенно важно для нагрузок с плохой локальностью (например, x264), где SHIP вместе с префетчером работает лучше, чем LRU.

Префетчинг помогает только в определённых случаях: если данные обладают плохой локальностью и кэш часто переполняется. Тогда предвыборка данных позволяет заранее подготовить то, что понадобится дальше, и уменьшить количество промахов. Однако если данные обладают хорошей локальностью (как в povray), префетчинг может даже ухудшить ситуацию, так как используются одни и те же данные, и предвыборка только искажает картину.

Во всех тестах видно, что с увеличением кэша hit rate растёт, но после определённого момента прирост становится меньше. Оптимальный размер зависит от типа нагрузки: для некоторых приложений достаточно среднего кэша, а для других нужен большой. Также оказалось, что выгоднее увеличивать количество путей (ways), чем сетов, при одинаковом общем размере кэша.

Таким образом, нельзя сказать, что одна конфигурация подходит всем. Нужно учитывать тип приложения, характер обращений к памяти и выбирать подходящую политику замещения и стратегию работы с кэшем.

Данные

Config	**Sets**	**Ways**	**ACCESS**	**HIT**	**MISS**	**hit rate (%)**	**Miss Rate (%)**
lru_no_prefetch	256	8	1,473,160	623,276	849,884	42.31	57.69
lru_no_prefetch	512	8	1,473,126	742,264	730,862	50.39	49.61
lru_no_prefetch	512	16	1,473,132	906,839	566,293	61.56	38.44
lru_no_prefetch	1024	8	1,473,094	902,632	570,462	61.27	38.73
lru_no_prefetch	1024	16	1,473,133	1,065,825	407,308	72.35	27.65
ship_no_prefetch	256	8	1,466,212	535,037	931,175	36.49	63.51
ship_no_prefetch	512	8	1,468,757	594,852	873,905	40.50	59.50
ship_no_prefetch	512	16	1,468,758	619,494	849,264	42.18	57.82
ship_no_prefetch	1024	8	1,471,131	687,438	783,693	46.73	53.27
ship_no_prefetch	1024	16	1,471,086	699,499	771,587	47.55	52.45
ship_spp_dev_pf	256	8	3,445,460	830,302	2,615,158	24.10	75.90
ship_spp_dev_pf	512	8	4,874,486	1,580,187	3,294,299	32.42	67.58
ship_spp_dev_pf	512	16	5,306,961	2,367,923	2,939,038	44.64	55.36
ship_spp_dev_pf	1024	8	6,514,126	2,820,802	3,693,324	43.30	56.70
ship_spp_dev_pf	1024	16	6,705,294	3,330,180	3,375,114	49.66	50.34

Таблица 1: L2 Cache, 437.leslie3d-134B.champsimtrace.xz

Config	**Sets**	**Ways**	**ACCESS**	**HIT**	**MISS**	**hit rate (%)**	**Miss Rate (%)**
lru_no_prefetch	256	8	1126569	1080360	46209	95.90	4.10
lru_no_prefetch	512	8	1126551	1126424	127	99.99	0.01
lru_no_prefetch	512	16	1126551	1126442	109	99.99	0.01
lru_no_prefetch	1024	8	1126551	1126442	109	99.99	0.01
lru_no_prefetch	1024	16	1126551	1126442	109	99.99	0.01
ship_no_prefetch	256	8	1126598	1063419	63179	94.40	5.60
ship_no_prefetch	512	8	1126504	1125967	537	99.95	0.05
ship_no_prefetch	512	16	1126551	1126442	109	99.99	0.01
ship_no_prefetch	1024	8	1126551	1126442	109	99.99	0.01
ship_no_prefetch	1024	16	1126551	1126442	109	99.99	0.01
ship_spp_dev_pf	256	8	1739277	1486867	252410	85.49	14.51
ship_spp_dev_pf	512	8	1770517	1544474	226043	87.23	12.77
ship_spp_dev_pf	512	16	1766672	1545468	221204	87.50	12.50
ship_spp_dev_pf	1024	8	1764222	1543348	220874	87.48	12.52
ship_spp_dev_pf	1024	16	1766672	1545468	221204	87.50	12.50

Таблица 2: L2 Cache, 453.povray-252B.champsimtrace.xz

Config	**Sets**	**Ways**	**ACCESS**	**HIT**	**MISS**	**hit rate (%)**	**Miss Rate (%)**
lru_no_prefetch	256	8	37064	8882	28182	23.96	76.04
lru_no_prefetch	512	8	37064	9163	27901	24.72	75.28
lru_no_prefetch	512	16	37064	9183	27881	24.77	75.23
lru_no_prefetch	1024	8	37064	9274	27790	25.02	74.98
lru_no_prefetch	1024	16	37063	11774	25289	31.77	68.23
ship_no_prefetch	256	8	37064	3394	33670	9.16	90.84
ship_no_prefetch	512	8	37064	6903	30161	18.62	81.38
ship_no_prefetch	512	16	37063	12445	24618	33.58	66.42
ship_no_prefetch	1024	8	37063	12918	24145	34.85	65.15
ship_no_prefetch	1024	16	37063	20118	16945	54.28	45.72
ship_spp_dev_pf	256	8	90140	16185	73955	17.96	82.04
ship_spp_dev_pf	512	8	91660	27160	64500	29.63	70.37
ship_spp_dev_pf	512	16	92847	29947	62900	32.26	67.74
ship_spp_dev_pf	1024	8	98300	53301	44999	54.22	45.78
ship_spp_dev_pf	1024	16	98300	53301	44999	54.22	45.78

Таблица 3: L2 Cache, 625.x264_s-12B.champsimtrace.xz